

Bidirectional Latent Action Model: Training Algorithms

Author
Seoul National University
`angel0119@snu.ac.kr`

September 17, 2026

Abstract

This note specifies the training algorithms for the Bidirectional Latent Action Model (BLAM), a fold/unfold latent-action architecture for cloth manipulation. A single encoder infers a fold action z_{fold} from a state pair (s_t, s_{t+1}) ; a small operator g_ψ derives the unfold action z_{unfold} from it, a relationship enforced by *cyclosymmetry*; a shared decoder D_θ , run in both directions, is tied together by *cycle-consistency*; and the resulting latent actions are distilled into a single deployable policy π_ω . We give the consolidated Stage 1 representation-learning algorithm, the Stage 2 policy distillation algorithm, and standalone/subroutine forms of the cyclosymmetry and cycle-consistency losses used inside Stage 1.

1 Overview

The architecture has two training stages. Stage 1 (Algorithm 1) learns a cyclosymmetric, cycle-consistent latent action space from paired cloth states connected by a folding action and its inverse. Stage 2 (Algorithm 2) distills the frozen Stage 1 representations, together with real ground-truth manipulator-action labels, into a single lightweight policy π_ω that runs with no encoder or decoder at inference time. Algorithms 3 and 5 give standalone pretraining forms of the two structural losses used inside Stage 1, together with the exact subroutines (Algorithms 4 and 6) that Stage 1 calls.

2 Stage 1: Representation Learning

Algorithm 1 Bidirectional Latent Action Model — Stage 1 Training (Consolidated)

Require: Paired cloth-state dataset $\mathcal{D} = \{(s_t, s_{t+1})\}$

Require: Stochastic encoder E_ϕ (outputs $\mu_{\text{fold}}, \sigma_{\text{fold}}$), cyclosymmetric operator g_ψ , shared decoder D_θ

Require: Loss weights $\lambda_{\text{fold}}, \lambda_{\text{unfold}}, \lambda_{\text{cyclosym}}, \lambda_{\text{cycle}}, \lambda_{\text{KL}}$

Require: Verification tolerance τ_{cycle}

Require: Adam hyperparameters: learning rate α , β_1, β_2 , ϵ_{adam}

Ensure: Trained parameters ϕ, ψ, θ

```

1: Initialize  $\phi, \psi, \theta$ ; initialize Adam moment buffers  $m \leftarrow 0, v \leftarrow 0$ 
2: while not converged do
3:   Sample minibatch  $\{(s_t, s_{t+1})\} \sim \mathcal{D}$ 
4:    $(\mu_{\text{fold}}, \sigma_{\text{fold}}) \leftarrow E_\phi(s_t, s_{t+1})$  ▷ single encoder pass
5:    $\varepsilon \sim \mathcal{N}(0, I)$ ;  $z_{\text{fold}} \leftarrow \mu_{\text{fold}} + \sigma_{\text{fold}} \odot \varepsilon$  ▷ reparameterization
6:    $z_{\text{unfold}} \leftarrow g_\psi(z_{\text{fold}})$  ▷ derived, no second encoder pass
7:    $\hat{s}_{t+1} \leftarrow D_\theta(s_t, z_{\text{fold}})$  ▷ fold decode — computed once, reused below
8:    $\hat{s}_t \leftarrow D_\theta(s_{t+1}, z_{\text{unfold}})$  ▷ unfold decode, shared weights
9:    $\mathcal{L}_{\text{fold}} \leftarrow \|\hat{s}_{t+1} - s_{t+1}\|^2$ 
10:   $\mathcal{L}_{\text{unfold}} \leftarrow \|\hat{s}_t - s_t\|^2$ 
11:   $\mathcal{L}_{\text{cyclosym}} \leftarrow \text{CYCLOSMMETRYLOSS}(z_{\text{fold}}, z_{\text{unfold}}, \psi)$  ▷ Algorithm 4
12:   $\mathcal{L}_{\text{cycle}} \leftarrow \text{CYCLECONSISTENCYLOSS}(s_t, \hat{s}_{t+1}, z_{\text{unfold}}, \theta)$  ▷ Algorithm 6, reuses  $\hat{s}_{t+1}$ 
13:   $\mathcal{L}_{\text{KL}} \leftarrow D_{\text{KL}}(\mathcal{N}(\mu_{\text{fold}}, \sigma_{\text{fold}}^2) \parallel \mathcal{N}(0, I))$ 
14:   $\mathcal{L} \leftarrow \lambda_{\text{fold}}\mathcal{L}_{\text{fold}} + \lambda_{\text{unfold}}\mathcal{L}_{\text{unfold}} + \lambda_{\text{cyclosym}}\mathcal{L}_{\text{cyclosym}} + \lambda_{\text{cycle}}\mathcal{L}_{\text{cycle}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}}$ 
15:   $g \leftarrow \nabla_{\phi, \psi, \theta} \mathcal{L}$ 
16:   $m \leftarrow \beta_1 m + (1 - \beta_1)g$ ;  $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$ 
17:   $\hat{m} \leftarrow m / (1 - \beta_1^t)$ ;  $\hat{v} \leftarrow v / (1 - \beta_2^t)$ 
18:   $(\phi, \psi, \theta) \leftarrow (\phi, \psi, \theta) - \alpha \hat{m} / (\sqrt{\hat{v}} + \epsilon_{\text{adam}})$ 
19: end while
20: Verification (cycle closes on held-out states):
21:    $\Delta_{\text{cycle}} \leftarrow \mathbb{E}_{\mathcal{D}_{\text{val}}} [\|D_\theta(D_\theta(s_t, z_{\text{fold}}), z_{\text{unfold}}) - s_t\|]$ 
22:   assert  $\Delta_{\text{cycle}} < \tau_{\text{cycle}}$ 
23: return  $\phi, \psi, \theta$ 

```

3 Stage 2: Policy Distillation

Algorithm 2 Policy Distillation from Bidirectional Latent Actions

Require: Labeled paired dataset $\mathcal{D} = \{(s_t, s_{t+1}, a_t^{\text{fold}}, a_t^{\text{unfold}})\}$, where $a_t^{\text{fold}}, a_t^{\text{unfold}}$ are real ground-truth manipulator actions

Require: Frozen Stage 1 parameters ϕ, ψ, θ (encoder E_ϕ , operator g_ψ , decoder D_θ)

Require: Student policy π_ω (single network, $s \mapsto a$)

Require: Loss weights $\lambda_{\text{BC}}, \lambda_{\text{ground}}, \lambda_{\text{sup}}$

Require: Adam hyperparameters: learning rate α , β_1, β_2 , ϵ_{adam}

Ensure: Trained parameters ω

- 1: Initialize ω ; initialize Adam moment buffers $m \leftarrow 0$, $v \leftarrow 0$
- 2: **while** not converged **do**
- 3: Sample minibatch $\{(s_t, s_{t+1}, a_t^{\text{fold}}, a_t^{\text{unfold}})\} \sim \mathcal{D}$
- 4: **With frozen ϕ, ψ, θ (no gradient):**
- 5: $z_{\text{fold}} \leftarrow E_\phi(s_t, s_{t+1})$
- 6: $z_{\text{unfold}} \leftarrow g_\psi(z_{\text{fold}})$
- 7: **Student forward passes (both directions, one policy):**
- 8: $\hat{a}_t^{\text{fold}} \leftarrow \pi_\omega(s_t)$
- 9: $\hat{a}_t^{\text{unfold}} \leftarrow \pi_\omega(s_{t+1})$
- 10: **Distillation loss (latent targets from frozen Stage 1):**
- 11: $\mathcal{L}_{\text{BC}} \leftarrow \|\hat{a}_t^{\text{fold}} - z_{\text{fold}}\|^2 + \|\hat{a}_t^{\text{unfold}} - z_{\text{unfold}}\|^2$
- 12: **Grounding loss (frozen decoder checks the decoded action reproduces the target state):**
- 13: $\mathcal{L}_{\text{ground}} \leftarrow \|D_\theta(s_t, \hat{a}_t^{\text{fold}}) - s_{t+1}\|^2 + \|D_\theta(s_{t+1}, \hat{a}_t^{\text{unfold}}) - s_t\|^2$
- 14: **Supervised loss (real manipulator-action labels):**
- 15: $\mathcal{L}_{\text{sup}} \leftarrow \|\hat{a}_t^{\text{fold}} - a_t^{\text{fold}}\|^2 + \|\hat{a}_t^{\text{unfold}} - a_t^{\text{unfold}}\|^2$
- 16: $\mathcal{L} \leftarrow \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{ground}}\mathcal{L}_{\text{ground}} + \lambda_{\text{sup}}\mathcal{L}_{\text{sup}}$
- 17: $g \leftarrow \nabla_\omega \mathcal{L}$
- 18: $m \leftarrow \beta_1 m + (1 - \beta_1)g$; $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$
- 19: $\hat{m} \leftarrow m/(1 - \beta_1^t)$; $\hat{v} \leftarrow v/(1 - \beta_2^t)$
- 20: $\omega \leftarrow \omega - \alpha \hat{m}/(\sqrt{\hat{v}} + \epsilon_{\text{adam}})$
- 21: **end while**
- 22: **return** ω $\triangleright$ deployed policy: $s_t \mapsto a_t$, no encoder/decoder at inference

4 Latent Cyclosymmetry Learning

Algorithm 3 Latent Cyclosymmetry Learning (Standalone Pretraining)

Require: Paired cloth-state dataset $\mathcal{D} = \{(s_t, s_{t+1})\}$

Require: Encoder E_ϕ (frozen or co-trained), cyclosymmetric operator g_ψ

Require: Loss weight $\lambda_{\text{cyclosym}}$; non-triviality margin $\epsilon_{\min}$; involution tolerance τ

Require: Adam hyperparameters: learning rate α , β_1, β_2 , ϵ_{adam}

Ensure: Trained parameters ψ such that g_ψ is a non-trivial involution: $g_\psi(g_\psi(z_{\text{fold}})) \approx z_{\text{fold}}$ and $g_\psi(z_{\text{fold}}) \not\approx z_{\text{fold}}$

```

1: Initialize  $\psi$ ; initialize Adam moment buffers  $m \leftarrow 0$ ,  $v \leftarrow 0$ 
2: while not converged do
3:   Sample minibatch  $\{(s_t, s_{t+1})\} \sim \mathcal{D}$ 
4:    $z_{\text{fold}} \leftarrow E_\phi(s_t, s_{t+1})$   $\triangleright$  stop-gradient into  $\phi$  if  $\phi$  is frozen
5:    $z_{\text{unfold}} \leftarrow g_\psi(z_{\text{fold}})$ 
6:    $z'_{\text{fold}} \leftarrow g_\psi(z_{\text{unfold}})$ 
7:    $\mathcal{L}_{\text{cyclosym}} \leftarrow \|z'_{\text{fold}} - z_{\text{fold}}\|^2$   $\triangleright$  symmetric, two-sided: gradients flow through both branches
8:    $g \leftarrow \nabla_\psi \lambda_{\text{cyclosym}} \mathcal{L}_{\text{cyclosym}}$ 
9:    $m \leftarrow \beta_1 m + (1 - \beta_1)g$ ;  $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$ 
10:   $\hat{m} \leftarrow m / (1 - \beta_1^t)$ ;  $\hat{v} \leftarrow v / (1 - \beta_2^t)$ 
11:   $\psi \leftarrow \psi - \alpha \hat{m} / (\sqrt{\hat{v}} + \epsilon_{\text{adam}})$ 
12: end while
13: Verification 1 (involution holds):
14:    $\Delta_{\text{inv}} \leftarrow \mathbb{E}_{\mathcal{D}_{\text{val}}} [\|g_\psi(g_\psi(z_{\text{fold}})) - z_{\text{fold}}\|]$ 
15:   assert  $\Delta_{\text{inv}} < \tau$ 
16: Verification 2 (non-trivial: rules out degenerate  $g_\psi \approx \text{id}$ ):
17:    $\Delta_{\text{triv}} \leftarrow \mathbb{E}_{\mathcal{D}_{\text{val}}} [\|g_\psi(z_{\text{fold}}) - z_{\text{fold}}\|]$ 
18:   assert  $\Delta_{\text{triv}} > \epsilon_{\min}$ 
19: return  $\psi$ 

```

Algorithm 4 CYCLOSMMETRYLOSS Subroutine (called from Algorithm 1)

```

1: function CYCLOSMMETRYLOSS( $z_{\text{fold}}, z_{\text{unfold}}, \psi$ )  $\triangleright z_{\text{unfold}}$  is already computed at Alg. 1, line 6 — not recomputed here
2:    $z'_{\text{fold}} \leftarrow g_\psi(z_{\text{unfold}})$   $\triangleright$  second application of  $g_\psi$ , reuses the same  $\psi$  being jointly trained
3:   return  $\|z'_{\text{fold}} - z_{\text{fold}}\|^2$   $\triangleright$  symmetric: gradients flow through both branches into  $\psi$  and, via  $z_{\text{unfold}}$ , into  $\phi$ 
4: end function

```

5 Cycle Consistency Algorithm

Algorithm 5 Cycle Consistency Learning (Standalone Pretraining)

Require: Paired cloth-state dataset $\mathcal{D} = \{(s_t, s_{t+1})\}$

Require: Encoder E_ϕ , cyclosymmetric operator g_ψ (frozen or co-trained), shared decoder D_θ

Require: Loss weight λ_{cycle} ; cycle-error tolerance τ_{cycle}

Require: Adam hyperparameters: learning rate α , β_1, β_2 , ϵ_{adam}

Ensure: Trained parameters θ such that folding then unfolding returns the original state:

$$D_\theta(D_\theta(s_t, z_{\text{fold}}), z_{\text{unfold}}) \approx s_t$$

- 1: Initialize θ ; initialize Adam moment buffers $m \leftarrow 0$, $v \leftarrow 0$
 - 2: **while** not converged **do**
 - 3: Sample minibatch $\{(s_t, s_{t+1})\} \sim \mathcal{D}$
 - 4: $z_{\text{fold}} \leftarrow E_\phi(s_t, s_{t+1})$ $\triangleright$ stop-gradient into ϕ if ϕ is frozen
 - 5: $z_{\text{unfold}} \leftarrow g_\psi(z_{\text{fold}})$ $\triangleright$ stop-gradient into ψ if ψ is frozen
 - 6: $\tilde{s}_{t+1} \leftarrow D_\theta(s_t, z_{\text{fold}})$ $\triangleright$ forward: fold
 - 7: $\bar{s}_t \leftarrow D_\theta(\tilde{s}_{t+1}, z_{\text{unfold}})$ $\triangleright$ backward: unfold the folded prediction
 - 8: $\mathcal{L}_{\text{cycle}} \leftarrow \|\bar{s}_t - s_t\|^2$
 - 9: $g \leftarrow \nabla_\theta \lambda_{\text{cycle}} \mathcal{L}_{\text{cycle}}$
 - 10: $m \leftarrow \beta_1 m + (1 - \beta_1)g$; $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$
 - 11: $\hat{m} \leftarrow m / (1 - \beta_1^t)$; $\hat{v} \leftarrow v / (1 - \beta_2^t)$
 - 12: $\theta \leftarrow \theta - \alpha \hat{m} / (\sqrt{\hat{v}} + \epsilon_{\text{adam}})$
 - 13: **end while**
 - 14: **Verification (cycle closes on held-out states):**
 - 15: $\Delta_{\text{cycle}} \leftarrow \mathbb{E}_{\mathcal{D}_{\text{val}}} [\|D_\theta(D_\theta(s_t, z_{\text{fold}}), z_{\text{unfold}}) - s_t\|]$
 - 16: **assert** $\Delta_{\text{cycle}} < \tau_{\text{cycle}}$
 - 17: **return** θ
-

Algorithm 6 CYCLECONSISTENCYLOSS Subroutine (de-duplicated; called from Algorithm 1)

- 1: **function** CYCLECONSISTENCYLOSS($s_t, \hat{s}_{t+1}, z_{\text{unfold}}, \theta$) $\triangleright \hat{s}_{t+1}$ passed in, not recomputed — caller already has it
 - 2: $\bar{s}_t \leftarrow D_\theta(\hat{s}_{t+1}, z_{\text{unfold}})$ $\triangleright$ only new decoder call in this subroutine
 - 3: **return** $\|\bar{s}_t - s_t\|^2$
 - 4: **end function**
-